

# Rapport Sécurité : SR10

Tests de sécurité : Failles corrigées et analyse des risques

Projet *Landr* : Plateforme de recrutement

Mathis Millot & Romain Pierre

Juin 2026

## Table des matières

|                                                            |          |
|------------------------------------------------------------|----------|
| <b>Introduction : Démarche et fichier de test .....</b>    | <b>2</b> |
| <b>(a) Contrôle d'accès : élévation de privilèges.....</b> | <b>2</b> |
| <b>(b) Force brute : limitation des tentatives.....</b>    | <b>3</b> |
| <b>(c) Injection SQL : requêtes paramétrées.....</b>       | <b>4</b> |
| <b>(d) Protection CSRF : jeton anti-falsification.....</b> | <b>5</b> |
| <b>(e) En-têtes HTTP de sécurité : helmet.....</b>         | <b>6</b> |
| <b>Bilan .....</b>                                         | <b>6</b> |

## Introduction : Démarche et fichier de test

---

Ce rapport documente les failles de sécurité identifiées et corrigées dans la plateforme *Landr*, ainsi que la suite de tests automatisés qui valide ces corrections. L'ensemble est regroupé dans `test/securite.test.js` : 15 tests répartis en cinq familles correspondant aux principaux risques d'une application web manipulant des comptes et des données personnelles (*Broken Access Control*, *Identification and Authentication Failures*, *Injection*, *CSRF* et *Security Misconfiguration*).

Comme les autres suites du projet, les tests s'appuient sur `supertest` et sur la vraie base de données. Trois utilisateurs (candidat, recruteur, admin) sont créés en `beforeAll` avec un email horodaté unique, via `utilisateur.create` puis `changeRole`, et supprimés en `afterAll` dans l'ordre des clés étrangères.

Pour chaque famille, on présente ci-dessous : la **faille** d'origine, le **risque** concret en l'absence de correctif, le **correctif** mis en place et le **test** qui le verrouille contre toute régression.

### (a) Contrôle d'accès : élévation de privilèges

---

**Faille.** Les routes sensibles (`/admin`, `/recruteur`, espaces candidat) n'étaient protégées que par la présence d'une session, sans vérification du rôle réel de l'utilisateur. Un compte candidat pouvait ainsi atteindre des écrans d'administration.

**Risque.** *Broken Access Control* : un utilisateur authentifié de bas privilège peut consulter ou modifier des ressources réservées à l'administrateur (validation d'organisations, changement de rôle, gestion des comptes). C'est une élévation horizontale et verticale de privilèges, l'une des failles web les plus critiques.

**Correctif.** Trois middlewares `isAdmin` / `isRecruteur` / `isCandidat` (`routes/index.js`) rechargent le rôle depuis la base à chaque requête (et non depuis la session, potentiellement périmée) puis renvoient un 403 en cas de rôle insuffisant, ou une redirection si la session est absente :

```
1 async function isAdmin(req, res, next) {
2   if (!req.session.user) return res.redirect('/connection');
3   const user = await refreshSessionUser(req);
4   if (!user || user.gone || user.inactive)
5     return req.session.destroy(() => res.redirect('/connection'));
6   if (user.role === 'Admin') return next();
7   return res.status(403).send('Accès refusé');
8 }
```

**Tests.** On vérifie les trois situations : sans session (redirection), rôle insuffisant (403), et confinement du recruteur hors de l'espace candidat (redirection) :

```
1 test('Sans session : GET /admin redirige vers /connection', async () =>
2   {
3     const res = await request(app).get('/admin');
4     expect(res.statusCode).toBe(302);
5     expect(res.headers.location).toBe('/connection');
6   });
7 test('Candidat connecté : GET /admin répond 403', async () => {
```

```

8   const res = await candidatAgent.get('/admin');
9   expect(res.statusCode).toBe(403);
10  });

```

Les autres tests vérifient symétriquement qu'un candidat est refusé sur `/recruteur` et qu'un recruteur est confiné hors de l'espace candidat. Recharger le rôle à chaque requête plutôt que de faire confiance à la session bloque aussi le cas d'un utilisateur rétrogradé ou désactivé pendant que sa session est encore ouverte.

## (b) Force brute : limitation des tentatives

**Faille.** La route `POST /login` acceptait un nombre illimité de tentatives de connexion, sans aucun ralentissement.

**Risque.** Attaque par force brute / *credential stuffing* : un attaquant peut tester des milliers de mots de passe par minute jusqu'à compromettre un compte. La protection `bcrypt` sur les mots de passe stockés ne suffit pas si l'on autorise un volume illimité d'essais en ligne.

**Correctif.** Un `rate-limiter` en mémoire (`routes/index.js`) bloque une IP après `LOGIN_MAX = 5` tentatives *échouées* sur une fenêtre glissante de 15 minutes (`isRateLimited` / `recordFailedLogin`). Point important : seules les tentatives échouées sont comptées, et un succès remet le compteur à zéro (`resetLoginAttempts`). Un utilisateur légitime ne se verrouille donc jamais lui-même.

**Test.** Cinq tentatives échouées passent, la sixième est rejetée par une redirection portant `error=ratelimit`. Jest isolant les modules par fichier, la Map des compteurs repart bien de zéro pour ce test :

```

1 test('5 tentatives échouées, la 6e est bloquée (error=ratelimit)', async
   () => {
2   const badCredentials = { email: 'brute_${TS}@jest.local', mdp: '
     WrongMdp999!' };
3   for (let i = 0; i < 5; i++) {
4     await request(app).post('/login').type('form').send(badCredentials);
5   }
6   const res = await request(app).post('/login').type('form').send(
     badCredentials);
7   expect(res.statusCode).toBe(302);
8   expect(res.headers.location).toContain('error=ratelimit');
9 });

```

## (c) Injection SQL : requêtes paramétrées

**Faille.** Une construction de requêtes par concaténation de chaînes (`... WHERE email = ' + email + '`) aurait permis d'injecter du SQL via les champs de formulaire.

**Risque.** Injection SQL : avec un payload comme `' OR '1'='1`, un attaquant peut contourner l'authentification, lire l'intégralité de la table `Utilisateur` (`UNION SELECT`), voire altérer ou détruire des données. C'est une faille à impact maximal sur la confidentialité et l'intégrité de la base.

**Correctif.** Toutes les requêtes du modèle utilisent des *placeholders* ? ; le pilote MySQL échappe les valeurs. La vérification du mot de passe se fait ensuite par `bcrypt.compare`, jamais dans le SQL :

```
1 const [rows] = await db.query(  
2   'SELECT id_user, email, mdp FROM Utilisateur WHERE email = ?',  
3   [email]); // valeur paramétrée, jamais concaténé  
4   e  
5 if (!rows[0]) return null;  
6 const valide = await bcrypt.compare(mdp, rows[0].mdp); // hors SQL  
7 if (!valide) return null;
```

**Tests.** Trois payloads classiques sont envoyés dans le champ email. Aucun ne doit aboutir sur un espace authentifié ; la réponse reste une redirection vers `/connection` :

```
1 const payloads = [  
2   { label: "classique OR 1=1", email: "' OR '1'='1' " },  
3   { label: "commentaire SQL --", email: "admin'--" },  
4   { label: "UNION SELECT", email: "' UNION SELECT * FROM  
    Utilisateur--" },  
5 ];  
6 test.each(payloads)('Payload "$label" ne contourne pas l\'  
    authentification',  
7   async ({ email }) => {  
8     const res = await request(app).post('/login').type('form')  
9       .send({ email, mdp: 'WrongMdp999!' });  
10    expect(res.statusCode).toBe(302);  
11    expect(res.headers.location).not.toBe('/admin');  
12    expect(res.headers.location).toContain('/connection');  
13  });
```

## (d) Protection CSRF : jeton anti-falsification

**Faible.** Les formulaires POST (connexion, modification de profil, actions admin) n'embarquaient aucun jeton anti-CSRF ; n'importe quel site tiers pouvait soumettre une requête au nom d'un utilisateur connecté.

**Risque.** *Cross-Site Request Forgery* : un attaquant fait exécuter à la victime, à son insu, une action authentifiée (changer son email, supprimer une candidature, valider une organisation pour un admin). L'attaque exploite le cookie de session envoyé automatiquement par le navigateur.

**Correctif.** Le middleware `csrf({ cookie: false })` (`app.js`) ajoute un jeton à chaque formulaire et rejette par un 403 toute requête mutante sans jeton valide. Pour ne pas casser les suites de tests fonctionnels existantes, la protection est désactivée uniquement en `NODE_ENV='test'` ; la suite de sécurité recharge donc l'application en mode production via `jest.isolateModules` pour tester le comportement réel :

```
1 // recharge l'app en mode production (CSRF actif) le temps du test  
2 jest.isolateModules(() => { csrfApp = require('../app'); });  
3  
4 test('POST /login sans token CSRF → 403', async () => {  
5   const res = await request(csrfApp).post('/login').type('form')  
6     .send({ email: 'x@x.com', mdp: 'Wrong12345!' });  
7   expect(res.statusCode).toBe(403);
```

```
8 });
```

**Tests.** On vérifie aussi que le formulaire de connexion contient bien un champ `_csrf` et que le jeton émis est non-vidé (longueur > 20), garantissant qu'il est réellement généré et non un placeholder.

## (e) En-têtes HTTP de sécurité : helmet

**Faible.** Les réponses HTTP ne portaient aucun en-tête de sécurité, laissant le navigateur sans garde-fou contre plusieurs classes d'attaques côté client.

**Risque.** *Security Misconfiguration.* Sans ces en-têtes : `X-Content-Type-Options` absent autorise le *MIME-sniffing* (un fichier uploadé interprété comme script) ; `X-Frame-Options` absent permet le *clickjacking* via une `<iframe>` ; et l'absence de *Content-Security-Policy* ouvre la porte à l'exécution de scripts externes en cas de XSS.

**Correctif.** `helmet()` est activé dans `app.js` avec une CSP restrictive (`default-src 'self'`, `frameAncestors 'none'`), qui complète l'échappement automatique des templates EJS contre le XSS :

```
1 app.use(helmet({ contentSecurityPolicy: { directives: {
2   defaultSrc: ['self'], frameAncestors: ['none'],
3   imgSrc: ['self', 'data:'], /* ... */
4 }}}));
```

**Tests.** On contrôle la présence et le contenu des trois en-têtes clés sur une réponse réelle (`nosniff`, `X-Frame-Options`, `CSP`) :

```
1 test('Content-Security-Policy présente et restrictive', async () => {
2   const res = await request(app).get('/connection');
3   expect(res.headers['x-content-type-options']).toBe('nosniff');
4   expect(res.headers['content-security-policy']).toContain("default-src
   'self'");
5 });
```

## Bilan

| Faible           | Risque principal                 | Correctif                                  |
|------------------|----------------------------------|--------------------------------------------|
| Contrôle d'accès | Élévation de privilèges          | Middlewares de rôle (403)                  |
| Force brute      | Vol de compte                    | Rate-limiter login (5/15 min)              |
| Injection SQL    | Fuite / altération de la base    | Requêtes paramétrées + <code>bcrypt</code> |
| CSRF             | Action forcée à l'insu           | Jeton <code>csrf</code> (403 si absent)    |
| En-têtes HTTP    | Clickjacking, MIME-sniffing, XSS | <code>helmet</code> + <code>CSP</code>     |

La suite `securite.test.js` (15 tests, tous passants) transforme ces cinq corrections en garanties non-régressives : toute réintroduction d'une faille fait désormais échouer la CI. Les défenses se complètent en profondeur (contrôle de rôle côté serveur, limitation des tentatives, paramétrage SQL, jeton CSRF et durcissement des en-têtes), couvrant les principales catégories applicables à la plateforme.